UNIVERSIDADE PAULISTA – UNIP
CURSO DE CIÊNCIA DA COMPUTAÇÃO

HENRIQUE BOCCOLI DE OLIVEIRA

Desenvolvimento de um sistema de criptografia com rotores em Python

SÃO PAULO
2026

HENRIQUE BOCCOLI DE OLIVEIRA

Desenvolvimento de um sistema de criptografia com rotores em Python

Trabalho apresentado ao curso de Ciência da Computação da Universidade Paulista – UNIP,

SÃO PAULO

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de criptografia e descriptografia de mensagens em Python, inspirado no funcionamento da máquina Enigma. O Sistema foi implementado sem o uso de bibliotecas de criptografia, com o objetivo de estudar, na prática, o conceito de cifras baseadas em rotores. A partir de uma chave informada pelo usuário, o programa gera três rotores, cada um com uma permutação própria de um conjunto de 142 caracteres e uma posição inicial. A cada caractere processado os rotores giram, de modo que a substituição muda ao longo da mensagem. O processo é reversível, e a mesma chave permite recuperar o texto original. Ao longo do desenvolvimento, os principais problemas foram a criptografia inicial restrita às letras de A a Z, a repetibilidade dos resultados entre versões do Python, o tratamento de acentos e a validação das entradas. Foram escritos 15 testes automatizados, todos aprovados. O projeto tem finalidade exclusivamente educacional e não deve ser usado para proteger informações reais, pois apresenta limitações como a ausência de refletor e de painel de conexões e um espaço de chaves reduzido.

SUMÁRIO

1 INTRODUÇÃO 5
1.1 Justificativa 5
1.2 Organização do trabalho 6
2 OBJETIVOS 7
2.1 Objetivo geral 7
2.2 Objetivos específicos 7
3 FUNDAMENTAÇÃO TEÓRICA 8
3.1 A máquina Enigma 8
3.2 Cifras de substituição e rotores 8
3.3 Ferramentas utilizadas 9
4 METODOLOGIA 10
5 DESENVOLVIMENTO 11
5.1 Arquitetura do sistema 11
5.2 Conjunto de caracteres 11
5.3 Geração dos rotores a partir da chave 12
5.4 Rotação e substituição 12
5.5 Validação das entradas 13
5.6 Interface com o usuário 13
6 PROBLEMAS ENCONTRADOS E SOLUÇÕES 14
6.1 Criptografia restrita às letras 14
6.2 Reversibilidade 14
6.3 Repetibilidade dos resultados 15
6.4 Acentos e Unicode 15
6.5 Entradas inválidas 15
7 RESULTADOS E DISCUSSÃO 16
7.1 Exemplos de execução 16
7.2 Testes automatizados 16
7.3 Limitações 17
8 CONCLUSÃO 18
REFERÊNCIAS 19
APÊNDICE A – Código-fonte dos módulos principais 20

1 INTRODUÇÃO

A necessidade de manter o conteúdo de uma mensagem em segredo é muito antiga e acompanha o desenvolvimento das sociedades. A criptografia é a área que estuda as técnicas para transformar uma mensagem legível em uma forma que só pode ser compreendida por quem conhece o método e a chave utilizados (STALLINGS, 2015). Da cifra usada por generais da Antiguidade aos algoritmos que protegem hoje as transações bancárias, o princípio é o mesmo: quem conhece a chave consegue recuperar a mensagem, e quem não a conhece encontra apenas símbolos sem sentido (SINGH, 2001).

Entre os dispositivos criados para esse fim, a máquina Enigma ocupa um lugar especial. Utilizada pelas forças armadas da Alemanha antes e durante a Segunda Guerra Mundial, ela combinava componentes elétricos e mecânicos para cifrar mensagens de modo que a mesma letra fosse substituída de formas diferentes ao longo do texto (KAHN, 1996). O funcionamento baseava-se em discos giratórios, chamados rotores, cuja posição mudava a cada letra digitada. Por isso, a Enigma é um exemplo muito citado quando se quer explicar como uma cifra pode ser mais elaborada do que uma simples troca fixa de letras.

Este trabalho descreve o desenvolvimento de um programa em Python que implementa um sistema de criptografia inspirado na Enigma. O programa recebe uma mensagem e uma chave, cifra o texto por meio de rotores e permite recuperar a mensagem original quando a mesma chave é informada. Não houve a intenção de reproduzir com fidelidade a máquina histórica. O objetivo foi construir uma versão simplificada, que pudesse ser lida e compreendida por estudantes, sem o uso de bibliotecas de criptografia prontas.

A linguagem Python foi escolhida por ter sintaxe clara e por dispor, em sua biblioteca padrão, de tudo o que o projeto necessitava, inclusive um módulo para testes automatizados (PYTHON SOFTWARE FOUNDATION, [s. d.]).

1.1 Justificativa

Implementar uma cifra do zero obriga a pensar em detalhes que passam despercebidos quando se usa uma biblioteca pronta: como transformar uma chave em uma configuração, como garantir que o processo possa ser desfeito, como tratar

caracteres que não pertencem ao alfabeto. Esses problemas aparecem em qualquer sistema que manipule texto e, por isso, o projeto tem valor como exercício de programação, além de servir como introdução prática à criptografia.

Há também um interesse histórico. O esforço para quebrar as mensagens da Enigma levou ao desenvolvimento de máquinas de cálculo especializadas em Bletchley Park e influenciou a criptoanálise que se praticou depois (SINGH, 2001).

1.2 Organização do trabalho

O trabalho está dividido em oito seções. A seção 2 apresenta os objetivos. A seção 3 traz a fundamentação teórica, com a história da Enigma e os conceitos de cifras de substituição e rotores. A seção 4 descreve a metodologia. A seção 5 detalha o desenvolvimento do sistema, e a seção 6 relata os problemas encontrados e as soluções adotadas. Os resultados e as limitações aparecem na seção 7, e a seção 8 reúne as conclusões. Ao final, o Apêndice A reproduz o código-fonte dos módulos principais.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver, em Python, um sistema didático de criptografia e descriptografia de mensagens, inspirado na máquina Enigma, no qual uma chave definida pelo usuário determine a configuração dos rotores.

2.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) implementar rotores capazes de realizar a substituição de caracteres e de girar a cada caractere processado;
b) derivar a configuração inicial dos rotores a partir da chave informada;
c) garantir que o processo seja reversível, de modo que a mesma chave recupere a mensagem original;
d) ampliar o conjunto de caracteres para incluir letras acentuadas, números e símbolos;
e) validar as entradas do usuário, evitando o encerramento inesperado do programa;
f) organizar o código em módulos e verificar o funcionamento por meio de testes automatizados.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 A máquina Enigma

A Enigma foi idealizada pelo engenheiro alemão Arthur Scherbius, que patenteou o projeto em 1918. Nos anos seguintes, versões comerciais foram oferecidas a empresas e, a partir do final da década de 1920, o exército e a marinha da Alemanha passaram a adotar modelos militares da máquina (KAHN, 1996).

Por fora, a máquina lembrava uma máquina de escrever, com um teclado, um painel de lâmpadas e uma caixa que abrigava os rotores. Ao pressionar uma tecla, o operador fechava um circuito elétrico. A corrente atravessava os rotores, cada um deles com ligações internas que trocavam uma letra por outra, e fazia acender no painel a lâmpada correspondente à letra cifrada (SINGH, 2001).

O ponto central do funcionamento é o giro. A cada tecla pressionada, o primeiro rotor avançava uma posição. Quando completava uma volta, fazia avançar o segundo, e este, ao completar a sua, o terceiro, de maneira semelhante ao odômetro de um automóvel. Como a configuração dos rotores mudava a cada letra, a mesma letra digitada duas vezes seguidas produzia resultados diferentes (SINGH, 2001).

Os modelos militares utilizavam três rotores, escolhidos entre um conjunto maior e colocados em uma ordem definida, além de um painel de conexões, que trocava pares de letras antes e depois da passagem pelos rotores. Havia ainda um refletor, que devolvia a corrente por um caminho diferente. O refletor tinha duas consequências importantes: a mesma configuração servia para cifrar e para decifrar, e nenhuma letra podia ser cifrada como ela mesma. Somando a ordem dos rotores, suas posições iniciais e as ligações do painel, o número de configurações possíveis chegava a cerca de 159 quintilhões (SINGH, 2001).

Apesar desse número, a Enigma foi quebrada. O primeiro grande avanço foi obtido por criptoanalistas poloneses, entre eles Marian Rejewski, ainda na década de 1930. Durante a guerra, o trabalho foi continuado em Bletchley Park, na Inglaterra, por uma equipe que incluía o matemático Alan Turing, com o auxílio de máquinas eletromecânicas projetadas para testar configurações (KAHN, 1996; SINGH, 2001). Entre os fatores que ajudaram os criptoanalistas estava justamente a propriedade de que nenhuma letra era cifrada como ela mesma, além de falhas no uso da máquina pelos operadores.

3.2 Cifras de substituição e rotores

Em uma cifra de substituição monoalfabética, cada letra do texto original é trocada sempre pela mesma letra cifrada. É o caso da cifra de César, em que o alfabeto é deslocado um número fixo de posições. Esse tipo de cifra é vulnerável à análise de frequência, pois as letras mais comuns de um idioma continuam aparecendo com a mesma proporção no texto cifrado (STALLINGS, 2015).

As cifras polialfabéticas procuram resolver esse problema usando vários alfabetos de substituição que se alternam ao longo da mensagem. O rotor da Enigma é uma forma mecânica de fazer isso. Cada rotor representa uma permutação do alfabeto e, a cada giro, essa permutação é aplicada com um deslocamento diferente, o que equivale a trocar de alfabeto. Com um único rotor de n posições, seriam n alfabetos antes de o padrão se repetir. Com três rotores girando como um odômetro, o padrão completo só se repete depois de n^3 caracteres.

Em termos matemáticos, seja P a permutação de um rotor, com P[i] indicando a saída para a entrada i, n o número de caracteres do alfabeto e p a posição atual do rotor. A saída do rotor para o índice i é dada pela equação 1. Na descriptografia, usa-se a permutação inversa Q, tal que Q[P[i]] = i, conforme a equação 2.

$$S(i) = (P[(i + p) \bmod n] - p) \bmod n \quad (1)$$

$$R(j) = (Q[(j + p) \bmod n] - p) \bmod n \quad (2)$$

A composição das saídas dos três rotores, na ordem, resulta no caractere cifrado. Para desfazer a operação, aplica-se a função R de cada rotor na ordem contrária, o que garante a reversibilidade do processo, pois R(S(i)) = i para qualquer posição p.

3.3 Ferramentas utilizadas

O sistema foi escrito em Python, linguagem interpretada de uso geral. Além das estruturas básicas da linguagem, como listas, dicionários, strings, funções e classes, foram usados apenas dois módulos da biblioteca padrão: unicodedata, para a normalização de caracteres acentuados, e unittest, para a escrita dos testes (PYTHON SOFTWARE FOUNDATION, [s. d.]). Nenhuma biblioteca externa foi necessária, e nenhuma biblioteca de criptografia foi empregada.

A normalização de caracteres segue as formas definidas pelo padrão Unicode (UNICODE CONSORTIUM, [s. d.]), assunto retomado na seção 6.

4 METODOLOGIA

Trata-se de um trabalho de natureza aplicada, cujo produto é um programa de computador. O desenvolvimento seguiu uma abordagem incremental: uma primeira versão com as funções essenciais foi construída, testada e depois melhorada em uma segunda versão, à medida que as limitações eram identificadas. As etapas foram as seguintes:

a) levantamento de informações sobre o funcionamento da Enigma, em livros e documentação, para decidir quais elementos seriam mantidos e quais seriam simplificados;
b) definição dos requisitos do sistema: uso de chave definida pelo usuário, rotores com giro a cada caractere, processo reversível, preservação de espaços, validação das entradas e ausência de bibliotecas de criptografia;
c) implementação da primeira versão, restrita às 26 letras do alfabeto;
d) execução de testes e análise dos resultados, etapa em que se observou que números, acentos e pontuação permaneciam legíveis no texto cifrado;
e) implementação da segunda versão, com o conjunto de caracteres ampliado, e atualização dos testes;
f) redação da documentação do projeto e deste trabalho.

O programa foi executado e testado com Python 3.12, em ambiente de linha de comando. O código foi organizado em módulos pequenos e escrito de forma direta, com nomes de variáveis e funções em português. A verificação foi feita por testes automatizados, escritos com o módulo unittest e executados a cada alteração relevante.

Este documento foi formatado conforme a ABNT NBR 14724 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2024), e as referências seguem a ABNT NBR 6023 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2018).

5 DESENVOLVIMENTO

5.1 Arquitetura do sistema

O projeto foi organizado de modo que cada arquivo tenha uma responsabilidade. O arquivo main.py apenas chama o menu, e toda a lógica de criptografia fica fora dele. O Quadro 1 resume os módulos.

Quadro 1 – Módulos do projeto

Arquivo	Responsabilidade
main.py	Ponto de entrada do programa.
interface.py	Menu de texto e leitura dos dados digitados.
enigma.py	Conjunto de caracteres, classe Enigma e funções de criptografia.
rotor.py	Classe Rotor, com fiação, posição e giro.
gerador.py	Cálculo da semente e gerador pseudoaleatório.
validacao.py	Validação da chave e da mensagem e exceções próprias.
tests/test_enigma.py	Testes automatizados.

Fonte: Elaborado pelo autor (2026).

Os dois elementos centrais são as classes Rotor e Enigma. A primeira representa um único rotor e conhece apenas a sua fiação e a sua posição. A segunda reúne três rotores, controla o giro e percorre o texto caractere por caractere. Para um uso mais simples, o módulo enigma.py também oferece as funções criptografar e descriptografar, que recebem a mensagem e a chave e devolvem o resultado.

5.2 Conjunto de caracteres

O conjunto de caracteres cifrados é definido pela constante ALFABETO, montada a partir de cinco partes: as 26 letras maiúsculas e as 26 minúsculas, 24 letras acentuadas maiúsculas e as mesmas 24 minúsculas, os dez algarismos e 32 símbolos de pontuação e uso comum, o que totaliza 142 caracteres. Maiúsculas e minúsculas são tratadas como caracteres distintos, de modo que a caixa da letra também é cifrada.

Um dicionário chamado INDICES associa cada caractere à sua posição no ALFABETO, o que permite converter um caractere em número e voltar ao caractere

depois da passagem pelos rotores. Espaços, tabulações, quebras de linha e qualquer caractere fora do conjunto, como emojis, são copiados sem alteração e não provocam giro dos rotores. A preservação dos espaços atendeu a um dos requisitos do projeto e traz como consequência o fato de o tamanho das palavras continuar visível, ponto discutido na seção 7.3.

5.3 Geração dos rotores a partir da chave

A chave é validada antes de ser usada e deve ter de 3 a 32 caracteres, contendo apenas letras sem acento e números. Em seguida, a função gerar_semente percorre os caracteres da chave e combina seus códigos em um número inteiro de 32 bits. A cada caractere, o valor acumulado é multiplicado por 33 e somado ao código do caractere multiplicado pela sua posição na chave, de modo que chaves com os mesmos caracteres em ordens diferentes produzam sementes diferentes.

A semente alimenta um gerador pseudoaleatório congruencial linear, escrito no próprio projeto. Nesse tipo de gerador, cada valor é obtido a partir do anterior pela relação $x' = (a \cdot x + c) \bmod m$, em que m vale 2^{32} e as constantes a = 1664525 e c = 1013904223 são as apresentadas por Press et al. (1992). A teoria desse tipo de gerador é discutida por Knuth (1997).

Com esse gerador, o programa embaralha a lista dos índices de 0 a 141 pelo algoritmo de Fisher-Yates, descrito por Knuth (1997), e obtém assim a fiação de cada rotor. Em seguida, sorteia a posição inicial do rotor. O processo se repete três vezes, e os três rotores resultantes dependem exclusivamente da chave. A decisão de escrever um gerador próprio, em vez de usar o módulo random, está relacionada à repetibilidade dos resultados, assunto tratado na seção 6.3.

5.4 Rotação e substituição

A classe Rotor guarda a fiação, que é uma lista de números em que cada posição indica a saída correspondente à entrada, e calcula também a fiação inversa, usada na descriptografia. O atributo posicao indica quantas posições o rotor já avançou. O método avancar incrementa a posição e devolve verdadeiro quando o rotor completa uma volta, isto é, quando a posição volta a zero. Os métodos passar_frente e passar_tras aplicam as equações 1 e 2, respectivamente.

No método de giro da classe Enigma, o primeiro rotor avança sempre. Se ele completar uma volta, o segundo avança, e assim por diante. O laço é interrompido no primeiro rotor que não completa volta, o que reproduz o comportamento de odômetro descrito na seção 3.1. Como cada rotor tem 142 posições, o padrão de substituição só se repete após $142^3 = 2.863.288$ caracteres cifrados.

A Figura 1 mostra o método que cifra um caractere. Primeiro os rotores giram e depois o índice do caractere passa pelos rotores, do primeiro ao terceiro. Na descriptografia, o método correspondente percorre os rotores em ordem inversa e usa passar_tras.

Figura 1 – Método que cifra um caractere

```
def _cifrar_caractere(self, caractere):
    self._girar_rotores()
    indice = INDICES[caractere]
    for rotor in self.rotores:
        indice = rotor.passar_frente(indice)
    return ALFABETO[indice]
```

Antes de processar cada mensagem, os rotores são recolocados na posição inicial definida pela chave. Isso garante que a mesma mensagem, cifrada duas vezes pela mesma máquina, produza sempre o mesmo resultado, condição necessária para a descriptografia.

5.5 Validação das entradas

A validação foi implementada no módulo validacao.py, com duas exceções próprias, ChaveInvalidaError e MensagemInvalidaError. A chave é rejeitada quando não é um texto, quando está vazia, quando tem menos de 3 ou mais de 32 caracteres e quando contém espaços, acentos ou símbolos. A mensagem é rejeitada quando não é um texto ou quando está vazia ou formada somente por espaços.

A interface captura essas exceções, mostra a mensagem de erro ao usuário e volta ao menu. Também são tratadas a interrupção do programa pelo teclado e o fim da entrada de dados, de modo que o programa não seja encerrado de forma inesperada.

5.6 Interface com o usuário

A interface é um menu de texto com três opções: criptografar, descriptografar e sair. Nas duas primeiras, o programa solicita a mensagem e a chave e exibe o resultado. A lógica do menu está no módulo interface.py, separada da lógica de criptografia, o que permite testar a máquina sem depender da entrada de dados pelo teclado.

6 PROBLEMAS ENCONTRADOS E SOLUÇÕES

Esta seção reúne os principais problemas que apareceram durante o desenvolvimento e as soluções adotadas.

6.1 Criptografia restrita às letras

Na primeira versão, o alfabeto dos rotores tinha apenas as 26 letras de A a Z. Letras minúsculas eram convertidas para maiúsculas antes da cifragem e depois devolvidas à caixa original. Todo o restante, incluindo números, acentos e pontuação, era copiado sem alteração. Os testes mostraram que isso enfraquecia o resultado e revelava parte do conteúdo. Com a chave segredo, a mensagem “Ataque ao amanhecer às 05h, Coronel!” foi cifrada como “Fhsljw fa lszfenvye àt 05k, Eyufzcw!”, em que o acento de “às”, o número 05 e a pontuação permaneciam visíveis (Quadro 2).

Quadro 2 – Comparação entre as duas versões (chave segredo)

Versão	Caracteres cifrados	Resultado para a mensagem “Ataque ao amanhecer às 05h, Coronel!”
Primeira	26 (letras de A a Z)	Fhsljw fa lszfenvye àt 05k, Eyufzcw!
Segunda	142 (letras, acentos, números e símbolos)	)n).à` ÚH gÍ>Û#çÄ<s $M @JO9 NRf4%+Võ

Fonte: Elaborado pelo autor (2026).

A solução foi ampliar o alfabeto para 142 caracteres, incluindo letras acentuadas, números e símbolos. Como a classe Rotor calcula o tamanho da fiação a partir da lista recebida, a mudança ficou concentrada na definição do ALFABETO e no método que decide se um caractere deve ser cifrado. As letras maiúsculas e minúsculas passaram a ser símbolos distintos e, por isso, o tratamento de caixa que existia na primeira versão foi removido. Na segunda versão, todos os caracteres da mensagem do exemplo, com exceção dos espaços, aparecem alterados no resultado.

6.2 Reversibilidade

A descriptografia precisava desfazer exatamente o que a criptografia fazia, inclusive o efeito do giro. Como o projeto não usa refletor, não é possível reaproveitar o mesmo caminho na ida e na volta. A solução foi fazer cada rotor guardar, além da fiação, a sua tabela inversa, montada uma única vez na criação do objeto, e percorrer os rotores do último para o primeiro na descriptografia. Como o giro depende apenas do número de caracteres já processados, e não do conteúdo, os rotores chegam a cada caractere exatamente na mesma posição nos dois processos. Os testes de ida e volta confirmam essa propriedade, inclusive para mensagens longas, com mais de uma volta completa do primeiro rotor.

6.3 Repetibilidade dos resultados

Uma alternativa simples seria usar o módulo random para embaralhar os rotores, tendo a chave como semente. A documentação da linguagem informa, porém, que a maior parte dos algoritmos e das funções de semente desse módulo pode mudar entre versões do Python (PYTHON SOFTWARE FOUNDATION, [s. d.]). Nesse caso, uma mensagem cifrada em uma versão poderia não ser decifrada em outra, mesmo com a chave correta. Para evitar esse risco, e também para cumprir o requisito de que o algoritmo fosse implementado no próprio projeto, foi escrito o gerador congruencial descrito na seção 5.3, cujo comportamento depende apenas de operações aritméticas com números inteiros.

6.4 Acentos e Unicode

Ao ampliar o alfabeto, surgiu um problema relacionado à representação dos acentos. No padrão Unicode, uma letra como “ã” pode ser representada por um único caractere ou pela combinação da letra “a” com o sinal de til (UNICODE CONSORTIUM, [s. d.]). As duas formas são idênticas quando exibidas na tela, mas são sequências diferentes de códigos. Sem tratamento, a segunda forma não seria reconhecida como pertencente ao alfabeto, e a letra seguiria sem cifrar, com o sinal de acento tratado como um caractere separado.

A solução foi normalizar o texto para a forma NFC, na qual as letras acentuadas são representadas por um único caractere, antes do processamento, por meio da função normalize do módulo unicodedata. Uma consequência é que a mensagem recuperada na descriptografia está sempre na forma NFC, o que não altera a aparência do texto.

6.5 Entradas inválidas

Um dos requisitos definidos era que entradas inválidas não deveriam encerrar o programa. Chaves vazias, curtas, longas ou com caracteres não permitidos, bem como mensagens vazias, levantam exceções específicas, que são capturadas na interface e exibidas como mensagens de erro. Os testes verificam nove tipos de chave inválida e quatro de mensagem inválida.

7 RESULTADOS E DISCUSSÃO

7.1 Exemplos de execução

A Figura 2 mostra uma sessão do programa. A mensagem contém letras maiúsculas e minúsculas, acentos, números e pontuação, e foi cifrada com a chave 1945. Na descriptografia, com a mesma chave, o texto original foi recuperado sem nenhuma diferença.

Figura 2 – Sessão de criptografia e descriptografia no terminal

```
Opção: 1
Mensagem: Atenção: reunião às 15h, código 4521!
Chave (3 a 32 letras/números): 1945

Resultado: l}áïú>Âp ü2F;ö`r x_ )rTù ù'4ÖÂú ï.VUÛ

Opção: 2
Mensagem criptografada: l}áïú>Âp ü2F;ö`r x_ )rTù ù'4ÖÂú ï.VUÛ
Chave (3 a 32 letras/números): 1945

Resultado: Atenção: reunião às 15h, código 4521!
```

O Quadro 3 apresenta outros exemplos. Ao trocar a chave de 1945 para 1946, o resultado da mesma mensagem mudou por completo, o que mostra que a chave determina a configuração dos rotores. A repetição de uma mesma letra, por sua vez, produz caracteres diferentes a cada posição, como se vê na última linha, efeito direto do giro dos rotores.

Quadro 3 – Exemplos de mensagens cifradas

Mensagem	Chave	Resultado
BOM DIA	1234	l|c @ëÕ
BOM DIA	4321	Tü x!z.
AAAAAAAAAA	1324	lMcÂWXâdú|

7.2 Testes automatizados

Foram escritos 15 testes com o módulo unittest, todos aprovados na versão final. Eles cobrem os seguintes pontos:

a) a ida e volta da criptografia, com a mesma chave;
b) a diferença entre os resultados de chaves distintas e a impossibilidade de recuperar a mensagem com uma chave errada;
c) a preservação de espaços, tabulações e quebras de linha;
d) a criptografia de números, acentos e símbolos;
e) mensagens com letras maiúsculas e minúsculas;
f) a manutenção de caracteres fora do conjunto, como emojis;
g) a variação da substituição ao longo da mensagem;
h) mensagens longas, que exigem mais de uma volta completa dos rotores;
i) a reutilização de uma mesma máquina para mais de uma mensagem;
j) a ausência de caracteres repetidos no alfabeto;
k) chaves e mensagens inválidas.

Os testes permitem verificar rapidamente que uma alteração no conjunto de caracteres ou nos rotores não prejudica a reversibilidade.

7.3 Limitações

O sistema tem finalidade exclusivamente educacional e apresenta limitações que o tornam inadequado para proteger informações reais:

a) não há refletor nem painel de conexões, elementos que, na máquina original, ampliavam o número de configurações e, no caso do refletor, tornavam o processo simétrico. Sem ele, cifrar e decifrar são operações diferentes, o que facilita a explicação, mas permite que um caractere seja cifrado como ele mesmo;
b) o espaço de chaves é pequeno, pois a semente tem 32 bits e há, portanto, no máximo cerca de 4,29 bilhões de configurações iniciais distintas, quantidade que um computador comum consegue testar por força bruta;

c) o gerador congruencial linear é previsível e não foi projetado para uso criptográfico (STALLINGS, 2015);
d) espaços e quebras de linha não são cifrados, o que revela o tamanho das palavras e facilita a análise por quem interceptar a mensagem;
e) a cifra continua sujeita à análise de frequência e a ataques com texto claro conhecido, já que a estrutura do algoritmo é pública e os rotores são fixos durante toda a mensagem;
f) a chave aceita apenas letras sem acento e números, e a diferenciação entre maiúsculas e minúsculas pode causar erros de digitação.

A Enigma original, mesmo com todos os seus componentes, foi quebrada, o que reforça que um sistema com menos recursos, como este, não oferece proteção real.

8 CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um sistema de criptografia em Python inspirado na máquina Enigma. O programa cifra e decifra mensagens a partir de uma chave informada pelo usuário, com rotores que giram a cada caractere e alteram a substituição ao longo do texto. Os objetivos definidos foram atendidos: os rotores são gerados pela chave, o processo é reversível, o conjunto de caracteres inclui acentos, números e símbolos, as entradas são validadas e o código foi verificado por 15 testes automatizados.

Durante o desenvolvimento, o problema de maior efeito foi o da primeira versão, que cifrava apenas letras. Ele mostrou que uma decisão aparentemente pequena, como a definição do alfabeto, pode expor parte da mensagem, e que testar com exemplos realistas, com acentos e números, é tão importante quanto testar com texto simples. Os demais problemas, como a reversibilidade, a repetibilidade entre versões da linguagem e a normalização dos acentos, mostraram que detalhes de implementação têm influência direta sobre o funcionamento correto de uma cifra.

Também ficou claro o quanto o projeto se distancia de um sistema seguro. A ausência de refletor e de painel de conexões, o espaço de chaves reduzido e o gerador previsível impedem que o programa seja usado para proteger informações reais, e o próprio histórico da Enigma mostra que mesmo uma máquina mais complexa pôde ser quebrada. O valor do trabalho está no aprendizado sobre rotores, chaves e reversibilidade e no exercício de organizar um programa em módulos com testes.

Como trabalhos futuros, sugere-se a inclusão de um refletor e de um painel de conexões, a possibilidade de cifrar também os espaços, o uso de uma semente com mais bits derivada da chave e o desenvolvimento de uma interface gráfica. Outra possibilidade é implementar um ataque simples de análise de frequência contra a própria cifra, para demonstrar na prática as suas fragilidades.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. 4. ed. Rio de Janeiro: ABNT, 2024.

_____. **ABNT NBR 6023**: informação e documentação: referências: elaboração. 2. ed. Rio de Janeiro: ABNT, 2018.

KAHN, David. **The codebreakers**: the comprehensive history of secret communication from ancient times to the internet. Rev. ed. New York: Scribner, 1996.

KNUTH, Donald E. **The art of computer programming**: seminumerical algorithms. 3. ed. Reading: Addison-Wesley, 1997. v. 2.

PRESS, William H. *et al.* **Numerical recipes in C**: the art of scientific computing. 2. ed. Cambridge: Cambridge University Press, 1992.

PYTHON SOFTWARE FOUNDATION. **The Python standard library**. [S. l.]: Python Software Foundation, [s. d.]. Disponível em: https://docs.python.org/3/library/index.html. Acesso em: 30 set. 2026.

SINGH, Simon. **O livro dos códigos**: a ciência do sigilo: do antigo Egito à criptografia quântica. Tradução de Jorge Calife. Rio de Janeiro: Record, 2001.

STALLINGS, William. **Criptografia e segurança de redes**: princípios e práticas. 6. ed. São Paulo: Pearson Education do Brasil, 2015.

UNICODE CONSORTIUM. **Unicode Standard Annex #15**: Unicode normalization forms. [S. l.]: Unicode Consortium, [s. d.]. Disponível em: https://unicode.org/reports/tr15/. Acesso em: 30 set. 2026.

APÊNDICE A – Código-fonte dos módulos principais

Este apêndice reproduz os módulos rotor.py, gerador.py e enigma.py. Os demais arquivos do projeto não foram incluídos por serem mais curtos e sem relação direta com o algoritmo.

rotor.py

```
class Rotor:
    def __init__(self, fiacao, posicao=0):
        self.fiacao = list(fiacao)
        self.tamanho = len(self.fiacao)
        self.inversa = [0] * self.tamanho
        for entrada, saida in enumerate(self.fiacao):
            self.inversa[saida] = entrada
        self.posicao = posicao % self.tamanho

    def avancar(self):
        self.posicao = (self.posicao + 1) % self.tamanho
        return self.posicao == 0

    def passar_frente(self, indice):
        entrada = (indice + self.posicao) % self.tamanho
        saida = self.fiacao[entrada]
        return (saida - self.posicao) % self.tamanho

    def passar_tras(self, indice):
        entrada = (indice + self.posicao) % self.tamanho
        saida = self.inversa[entrada]
        return (saida - self.posicao) % self.tamanho
```

gerador.py

```
MODULO = 2 ** 32


def gerar_semente(chave):
    semente = 5381
    for posicao, caractere in enumerate(chave, start=1):
        semente = (semente * 33 + ord(caractere) * posicao) % MODULO
    return semente


class GeradorPseudoaleatorio:
    def __init__(self, semente):
        self.estado = semente

    def proximo(self):
        self.estado = (self.estado * 1664525 + 1013904223) % MODULO
```

```
        return self.estado >> 8

    def inteiro_ate(self, limite):
        return self.proximo() % limite

    def embaralhar(self, itens):
        itens = list(itens)
        for i in range(len(itens) - 1, 0, -1):
            j = self.inteiro_ate(i + 1)
            itens[i], itens[j] = itens[j], itens[i]
        return itens
```

enigma.py

```
import unicodedata

from gerador import GeradorPseudoaleatorio, gerar_semente
from rotor import Rotor
from validacao import validar_chave, validar_mensagem

LETRAS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
ACENTUADAS = "ÁÀÂÃÄÉÈÊËÍÌÎÏÓÒÔÕÖÚÙÛÜÇÑ"
DIGITOS = "0123456789"
SIMBOLOS = ".,;:!?-_'\"()[]{}@#$%&*+=/\\<>|~^`"

ALFABETO = (
    LETRAS
    + LETRAS.lower()
    + ACENTUADAS
    + ACENTUADAS.lower()
    + DIGITOS
    + SIMBOLOS
)
INDICES = {caractere: indice for indice, caractere in enumerate(ALFABETO)}
QUANTIDADE_ROTORES = 3


class Enigma:
    def __init__(self, chave):
        self.chave = validar_chave(chave)
        self.rotores = self._montar_rotores()
        self.posicoes_iniciais = [rotor.posicao for rotor in self.rotores]

    def _montar_rotores(self):
        gerador = GeradorPseudoaleatorio(gerar_semente(self.chave))
        rotores = []
        for _ in range(QUANTIDADE_ROTORES):
            fiacao = gerador.embaralhar(range(len(ALFABETO)))
            posicao = gerador.inteiro_ate(len(ALFABETO))
            rotores.append(Rotor(fiacao, posicao))
        return rotores
```

```
    def _reiniciar(self):
        for rotor, posicao in zip(self.rotores, self.posicoes_iniciais):
            rotor.posicao = posicao

    def _girar_rotores(self):
        for rotor in self.rotores:
            if not rotor.avancar():
                break

    def _cifrar_caractere(self, caractere):
        self._girar_rotores()
        indice = INDICES[caractere]
        for rotor in self.rotores:
            indice = rotor.passar_frente(indice)
        return ALFABETO[indice]

    def _decifrar_caractere(self, caractere):
        self._girar_rotores()
        indice = INDICES[caractere]
        for rotor in reversed(self.rotores):
            indice = rotor.passar_tras(indice)
        return ALFABETO[indice]

    def _processar(self, texto, transformar_caractere):
        self._reiniciar()
        texto = unicodedata.normalize("NFC", texto)
        resultado = []
        for caractere in texto:
            if caractere in INDICES:
                resultado.append(transformar_caractere(caractere))
            else:
                resultado.append(caractere)
        return "".join(resultado)

    def criptografar(self, mensagem):
        validar_mensagem(mensagem)
        return self._processar(mensagem, self._cifrar_caractere)

    def descriptografar(self, mensagem):
        validar_mensagem(mensagem)
        return self._processar(mensagem, self._decifrar_caractere)


def criptografar(mensagem, chave):
    return Enigma(chave).criptografar(mensagem)


def descriptografar(mensagem, chave):
    return Enigma(chave).descriptografar(mensagem)
```